

Object Recognition

Dr. Ei Moh Moh Aung
FCS



Object Recognition

- Object recognition in image processing refers to the task of **identifying** and **classifying** objects within digital images or video frames.
- It involves the use of computer vision techniques and machine learning algorithms to automatically **detect** and **recognize** objects of interest in visual data.

Object Recognition

- The goal of object recognition is to assign meaningful **labels** or **categories** to objects in an image.
- This can involve **identifying common objects** such as cars, people, animals, or specific object instances such as a particular breed of dog or a specific model of car.

Object Recognition

- Object recognition typically involves several steps, including:
 - **Preprocessing:** This step involves preparing the image data for analysis by performing operations such as **resizing**, **normalizing**, or **enhancing** the image.
 - **Feature Extraction:** Features that represent discriminative characteristics of objects are extracted from the preprocessed image. These features can be low-level visual descriptors like **edges**, **corners**, or **textures**, or **higher-level features** learned from deep neural networks.

Object Recognition

- **Classification or Detection:** The extracted features are used to classify the objects or detect their presence in the image. Classification involves **assigning a specific label or category** to an entire image, while detection involves **localizing** and **classifying** objects within an image.
- **Post-processing:** After classification or detection, post-processing techniques are applied to refine the results, such as **filtering** out false positives, **grouping** detections, or **estimating** object poses and attributes.

Image Processing

- We can distinguish between these three computer vision tasks:
 - **Image Classification:** **Predict** the type or class of an object in an image.
 - **Object Localization:** Locate the **presence** of objects in an image and indicate their location with a bounding box.
 - **Object Detection:** Locate the **presence of objects** with a bounding box and types or classes of the located objects in an image.

Image Processing

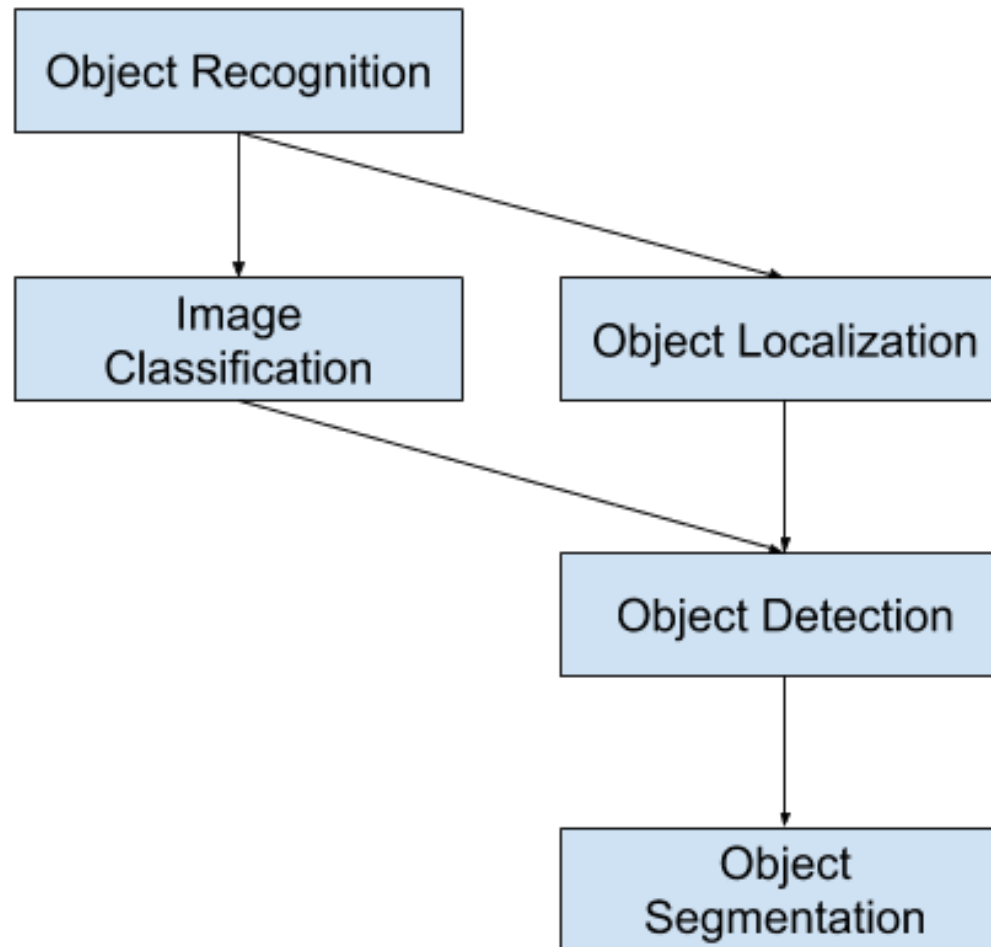


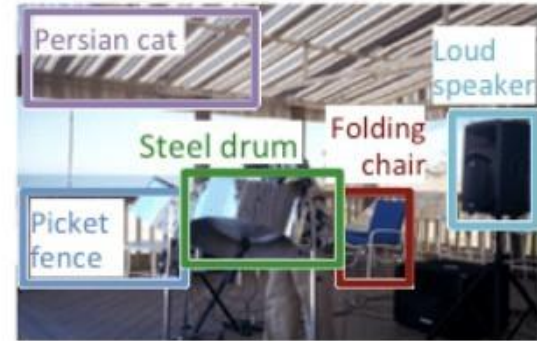
Image Processing

Single-object localization

Steel drum

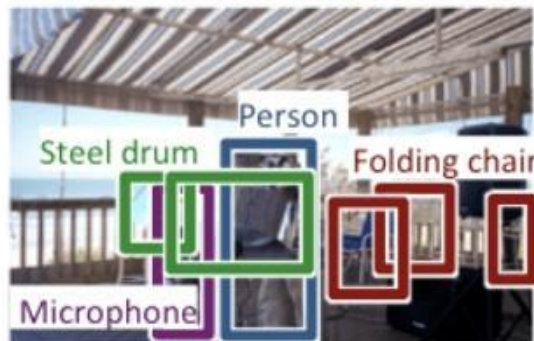


Ground truth

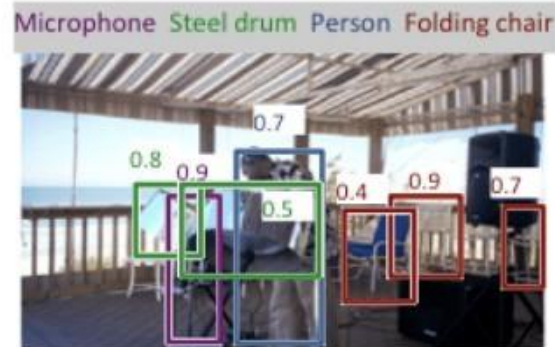


Accuracy: 1

Object detection



Ground truth



AP: 1.0 1.0 1.0 1.0

The Birth of R-CNN

- Authors:
 - Ross Girshick et al. (UC Berkeley), 2014.
- Context:
 - AlexNet (2012) revolutionized classification. R-CNN was the first successful attempt to apply CNNs to the more complex task of detection.
- Name:
 - "**Regions with CNN features**" because it applies a CNN to region proposals.
- Impact:
 - Improved mAP (mean Average Precision) on PASCAL VOC dataset from ~35% to ~53%.

The R-CNN Pipeline Overview

- The R-CNN approach consists of three main modules:
 - **Region Proposal Generation:** Find potential object locations ("blobs").
 - **Feature Extraction:** Use a CNN to extract a fixed-length feature vector from each region.
 - **Classification:** Use linear SVMs to classify each region.
 - (Optional Step 4: Bounding Box Regression)

The RCNN

R-CNN: *Regions with CNN features*

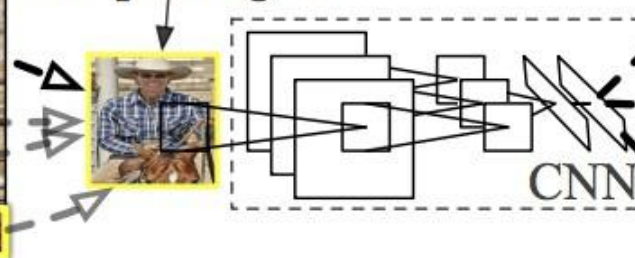


1. Input image

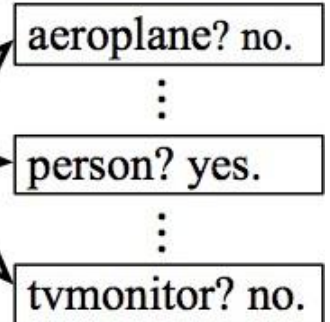


2. Extract region proposals (~2k)

warped region



3. Compute CNN features



4. Classify regions

Module 1: Region Proposals

- **Problem:** Searching every pixel window is computationally impossible.
- Solution: "Selective Search" algorithm.
- How it works:
 - Starts by **over-segmenting** the image based on color, texture, size, and shape similarity.
 - Greedily **merges similar regions** to form larger ones.
- Result: Generates $\sim 2,000$ category-independent region proposals per image.

Module 2: Feature Extraction (The CNN)

- Input:
 - The region proposals have different sizes, but CNNs (like AlexNet) need a fixed input size (e.g., 227x227).
- Warping:
 - Each region proposal is warped (resized/stretched) to fit the required **227x227 input size**, regardless of aspect ratio.
- Network:
 - A standard CNN (AlexNet) pre-trained on ImageNet is used.
- Output:
 - A **4096-dimensional feature vector** is extracted from the last fully connected layer (before the softmax).

Module 3: Classification

- Why SVMs?
 - At the time, training a softmax classifier end-to-end with the detection data was difficult due to limited data.
- Process:
 - Train one **binary Linear SVM per class** (e.g., Cat vs. Non-Cat, Car vs. Non-Car).
 - **Input**: The 4096-dimensional feature vector.
 - **Output**: A score indicating confidence that the region belongs to that class.

Module 4: Bounding Box Regression

- Problem:
 - The Selective Search proposal might not perfectly frame the object (e.g., might cut off the tail of a cat).
- Solution:
 - A **linear regression model** is trained to correct the coordinates.
- Input:
 - Feature vector of the region.
- Output:
 - **4 correction factors** (dx, dy, dw, dh) to **shift** and **resize** the bounding box closer to the Ground Truth.

Training Process: Phase 1 (Pre-training)

- Supervised Pre-training:
 - Train the CNN (AlexNet) on a large auxiliary dataset (ImageNet classification) with 1000 classes.
- Goal:
 - Learn **general feature extractors** (edges, textures, shapes).
- Labels:
 - Image-level labels only (no bounding boxes needed here).

Training Process: Phase 2 (Fine-tuning)

- **Domain-specific Fine-tuning:** Replace the 1000-way classification layer with a new (N+1)-way layer (N classes + Background).
- **Data:** Warped region proposals from the detection dataset (PASCAL VOC/COCO).
- **IoU Threshold:**
 - Positive: Regions with **IoU** (Intersection over Union) > 0.5 with a Ground Truth box.
 - Negative: Regions with $\text{IoU} < 0.5$ (Background).
- **Goal:** Adapt the CNN to detection tasks.

IoU (Intersection over Union)

- Definition:
 - A metric to measure the **accuracy of an object detector**.
- Formula:
 - **IoU**=Area of Union/Area of Intersection
- Usage:
 - Used to define positive and negative samples during training and to evaluate predictions during testing.

Object Classification with SVMs

- Why Separate Training?
 - The **SVMs are trained after the CNN features** are fixed.
- Hard Negative Mining:
 - Most of the 2,000 proposals are background (easy negatives).
 - SVMs are trained iteratively by focusing on "hard negatives" (background regions that look like objects) to improve robustness.

Test-Time Detection

- **Input Image.**
- Generate $\sim 2,000$ proposals (Selective Search). **Warp**
- **each proposal.**
- Run CNN forward pass for each proposal to get features. Score each proposal using all class-specific SVMs. Apply Non-Maximum Suppression (NMS).

Non-Maximum Suppression (NMS)

- Problem: The detector will find multiple overlapping boxes for the same object (e.g., 5 boxes around one cat).
- Solution:
 - Sort boxes by confidence score.
 - Pick the highest scoring box.
 - Discard all other boxes that have high overlap ($\text{IoU} > \text{threshold}$) with the chosen box.
 - Repeat until no boxes remain.

Challenges & Limitations: Speed

- Training is slow:
 - Must train CNN, then SVMs, then Regressors separately. (84 hours on 2014 hardware).
- Inference is slow:
 - Every single image requires running the CNN $\sim 2,000$ times (once per proposal).
- Time per image:
 - ~ 47 seconds on a GPU (unusable for real-time applications).

Storage & Complexity

- Disk Space:
 - Feature vectors for all proposals must be stored to disk to train the SVMs (hundreds of GBs).
- Multi-stage Pipeline:
 - Not end-to-end. Errors in Selective Search cannot be fixed by the CNN. Errors in the CNN cannot be fixed by the SVM.

The Path Forward

- R-CNN proved CNNs work for detection but was too slow.
- SPP-Net (Spatial Pyramid Pooling):
 - Solved the warping/fixed-size issue.
- Fast R-CNN:
 - Solved the speed issue by running the CNN once per image.
- Faster R-CNN:
 - Replaced slow Selective Search with a Region Proposal Network (RPN).

Image Segmentation

- This technique gives us a far more granular understanding of the object(s) in the image.
- The image shown below will help you to understand what image segmentation is:

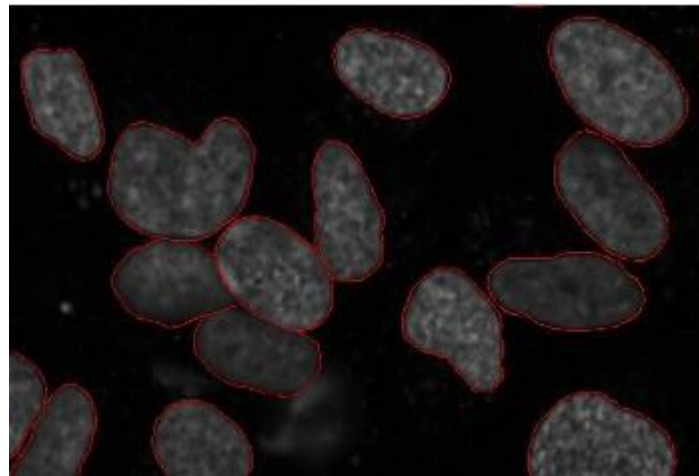
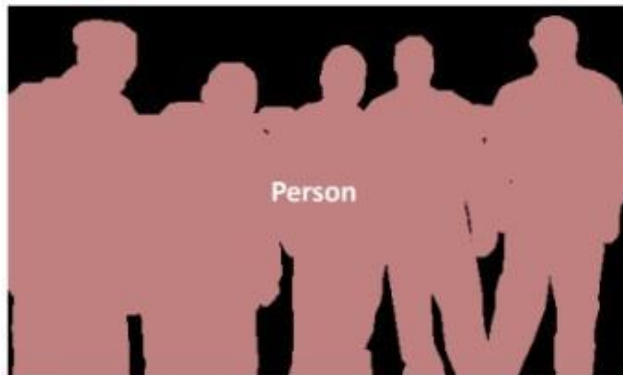
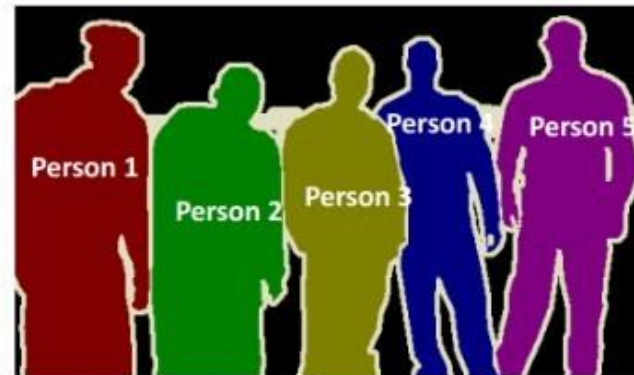


Image Segmentation

- All 5 objects in the left image are people. Hence, semantic segmentation will classify all the people as a single instance.
- Now, the image on the right also has 5 objects (all of them are people). But here, different objects of the same class have been assigned as different instances. This is an example of **instance segmentation**.



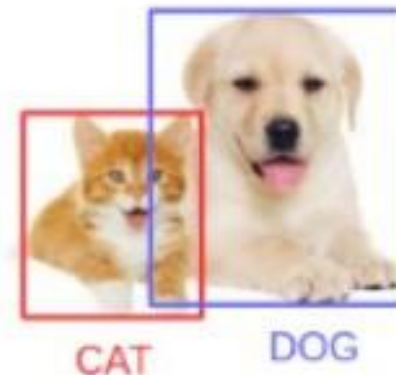
Semantic Segmentation



Instance Segmentation

Mask RCNN

- Mask R-CNN is basically an extension of Faster R-CNN. Faster R-CNN is widely used for object detection tasks.
- For a given image, it **returns the class label** and **bounding box** coordinates for each object in the image. So, let's say you pass the following image:



Mask RCNN

- Let's first quickly understand how Faster R-CNN works. This will help us grasp the intuition behind Mask R-CNN as well.
 - Faster R-CNN first uses a ConvNet to **extract feature maps** from the images
 - These feature maps are then passed through a Region Proposal Network (**RPN**) which returns the candidate **bounding boxes**
 - We then apply an **Roi pooling layer** on these candidate bounding boxes to bring all the candidates to **the same size**
 - And finally, the proposals are passed to a **fully connected layer** to classify and output the **bounding boxes** for objects

Mask RCNN

- **Backbone Model**
 - Similar to the ConvNet that we use in Faster R-CNN to extract feature maps from the image, we use the **ResNet 101** architecture to extract features from the images in Mask R-CNN.
 - So, the first step is to **take an image and extract features** using the ResNet 101 architecture. These features act as an input for the next layer.

Mask RCNN

- **Region Proposal Network (RPN)**
 - Now, we take the feature maps obtained in the previous step and apply a **region proposal network** (RPN).
 - This basically **predicts if an object is present** in that region (or not).
 - In this step, we get those regions or feature maps which the model predicts contain some object.

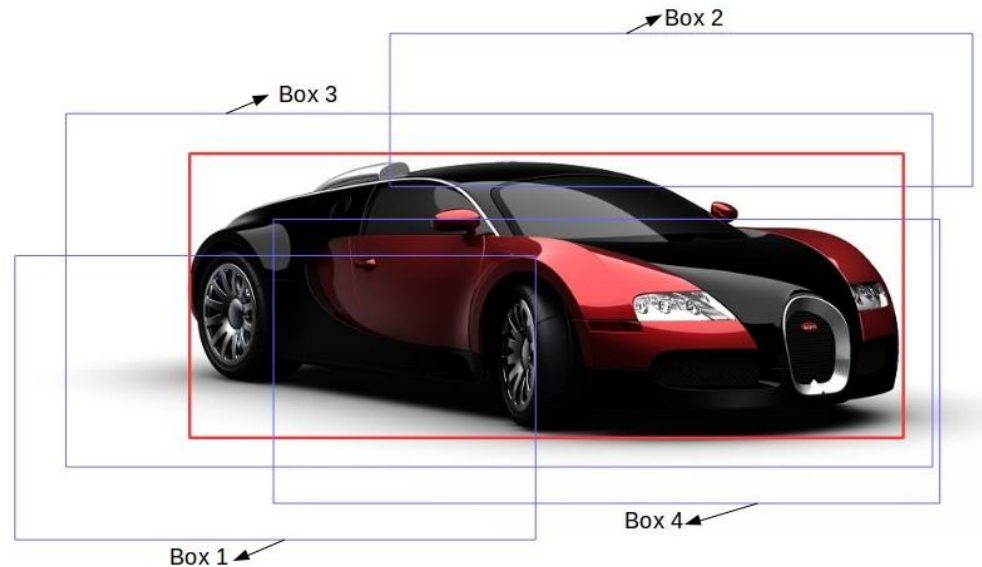
Mask RCNN: Region of Interest

- The regions obtained from the RPN might be of different shapes. Hence, we apply a pooling layer and **convert all the regions to the same shape**.
- Next, these regions are passed through a fully connected network so that the class **label** and bounding **boxes** are predicted.
- Till this point, the steps are almost similar to how Faster R-CNN works.
- Now comes the difference between the two frameworks. In addition to this, Mask R-CNN also generates the **segmentation mask**.

Mask RCNN: Region of Interest



Here, the IoU of **Box 1** and **Box 2** is possibly **less than 0.5**, whereas the IoU of **Box 3** and **Box 4** is approximately **greater than 0.5**. Hence, we can say that **Box 3** and **Box 4** are the region of interest for this particular image whereas **Box 1** and **Box 2** will be neglected.



Mask RCNN: Segmentation

- Once we have the Rols based on the IoU values, we can **add a mask branch** to the existing architecture.
- This returns the **segmentation mask** for each region that contains an object.
- It returns a **mask** of size **28 X 28** for each region which is then scaled up for inference.

Mask RCNN: Segmentation



COCO Dataset

- COCO is a **large-scale object detection, segmentation, and captioning dataset**. COCO has several features:
 - Object segmentation
 - Recognition in context
 - Superpixel stuff segmentation
 - 330K images (>200K labeled)
 - 1.5 million object instances
 - 80 object categories
 - 91 stuff categories
 - 5 captions per image
 - 250,000 people with keypoints

YOLO Models

- The YOLO model was first described by Joseph Redmon, et al. in the **2015** paper titled “*You Only Look Once: Unified, Real-Time Object Detection.*”
- Note that *Ross Girshick*, developer of R-CNN, was also an author and contributor to this work, then at *Facebook AI Research*.
- The approach involves a single neural network trained end to end that takes a photograph as input and **predicts bounding boxes** and class labels for each bounding box directly.
- The technique offers lower predictive accuracy (e.g. more localization errors), although operates at **45 frames per second** and up to **155 frames per second** for a speed-optimized version of the model.

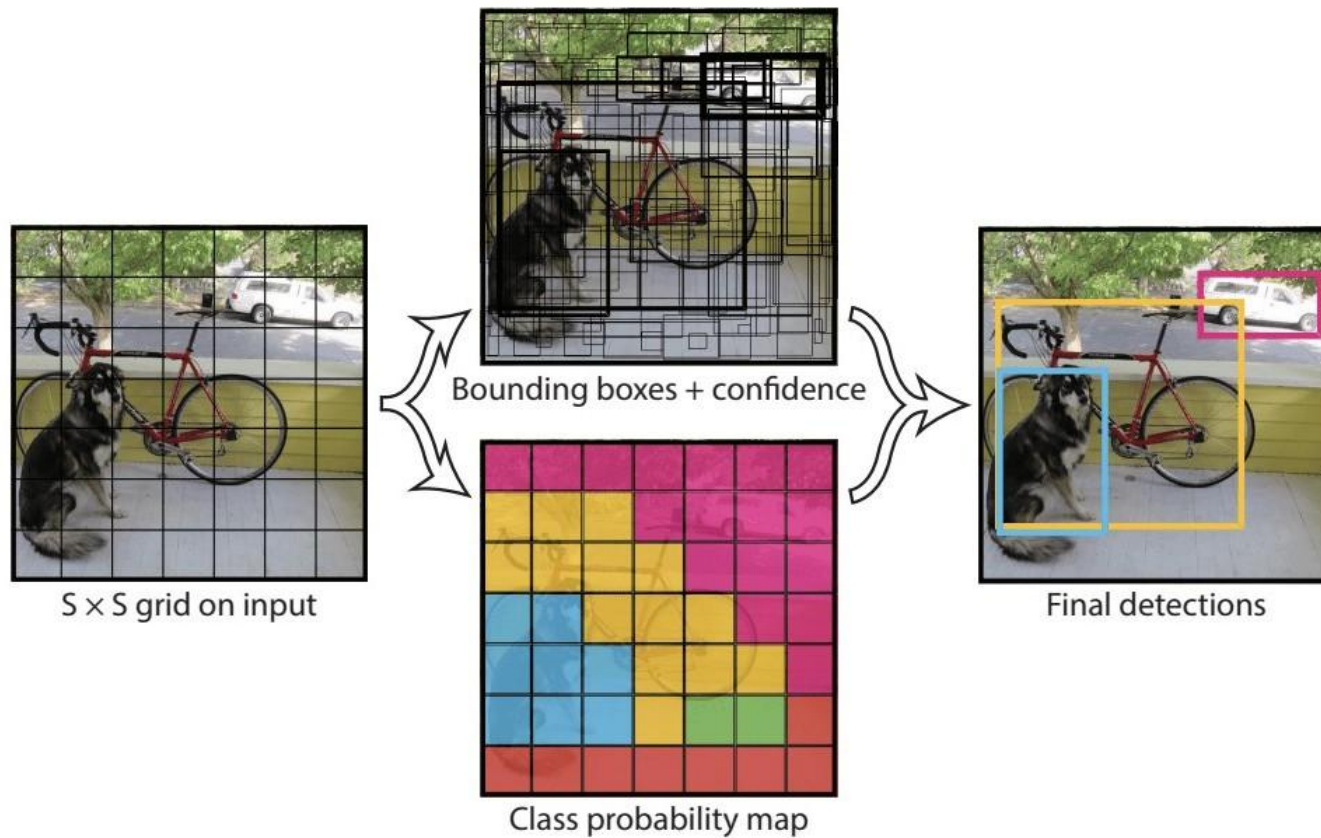
YOLO Models

- The model works by first splitting the input image into a **grid of cells**, where each cell is responsible for predicting a bounding box if the **center** of a bounding box falls within the cell.
- Each grid cell predicts a bounding box involving the x, y coordinate and the **width and height and the confidence**. A class prediction is also based on each cell.

YOLO Models

- For example, an image may be divided into a **7×7 grid** and each cell in the grid may **predict 2 bounding boxes**, resulting in 94 proposed bounding box predictions.
- The class probabilities map and the bounding boxes with confidences are then combined into a final set of **bounding boxes** and **class labels**.
- The image taken from the paper below summarizes the two outputs of the model.

YOLO Models



YOLO: Step by Step

- 1. Divide the image into a grid of cells.
 - The YOLO algorithm divides the image into a grid of cells, typically 7×7 or 13×13 .
 - Each cell is responsible for predicting a set of bounding boxes and class probabilities.

YOLO: Step by Step

- 2. Predict bounding boxes and class probabilities for each cell.
 - For each cell, the YOLO algorithm predicts a set of bounding boxes and class probabilities.
 - The **bounding boxes** are represented as four coordinates: the **top left corner**, the **bottom right corner**, and the **width and height** of the box.
 - The class probabilities represent the probability that the object in the box belongs to a particular class.

YOLO: Step by Step

- 3. Apply non-max suppression.
 - The bounding boxes predicted by the YOLO algorithm may **overlap**.
 - To remove overlapping boxes, the YOLO algorithm applies a **non-max suppression algorithm**.
 - This algorithm keeps the box with the **highest confidence score**, and it removes all other boxes that have a high overlap with the selected box.

YOLO: Step by Step

- 4. Draw the bounding boxes and class labels on the image.
 - The final step is to draw the bounding boxes and class labels on the image.
 - The bounding boxes are drawn in a different color for each class, and the class labels are displayed next to the bounding boxes.

YOLO: Summary

- Divide the image into a grid of cells.
- Predict bounding boxes and class probabilities for each cell.
- Apply non-max suppression.
- Draw the bounding boxes and class labels on the image.

Applications of Computer Vision

- Gender Prediction
- Image Mapping
- Face / Object Recognition
- Image Recreation
- Image Matching

END